Copilot Spaces and context

Give Copilot the right material, and only that

Session M2-03 | 3 hours

Agenda

Segment	Time
Context windows	8 min
Copilot Spaces	15 min
The other mechanisms	15 min
Which one when	8 min
Demo: same task, three contexts	9 min
Pros, cons, tips, mistakes	5 min

Context window

The **context window** is everything the model can see for one reply.

- your prompt and the chat so far
- instructions Copilot adds for you
- files, snippets, and Space sources
- tool results

It has a size limit. Anything outside it doesn't exist for the model.

Why curation matters

- **Missing context:** Copilot guesses. Guesses look confident.
- **Too much context:** the answer gets vague, slower, and more expensive.
- **Wrong context:** Copilot follows it faithfully.

Goal: the smallest set of sources that makes the task answerable.

What is a Copilot Space?

A named, saved **collection of context** that you ask questions against.

- **Sources:** files, folders, repositories, pull requests, issues, uploads, pasted text
- **Instructions:** a few lines on the Space's job and limits
- Lives on GitHub, outside any chat history

Reach it in Copilot Chat on github.com, or in an IDE or the CLI through the GitHub MCP server.

Sources and refresh

Source	How it loads
File	Whole file, in the window, every question
Repository	Searched. Only relevant parts come back
Pull request, issue	Linked by URL
Upload, pasted text	Static. It never updates

GitHub files follow the `main` **branch**. Push to `main` and the Space updates.

Who can see a Space

Owner	Sharing options
Personal	Private, specific people, or public (view-only)
Organization	Org members as admin, editor, or viewer, or no access

Viewers only see sources **they** can access. Sharing a Space doesn't open the repository.

Mixing private and multi-org sources can break chat. Check the current docs.

Spaces: where and what it costs

- Any Copilot plan, including Free
- Questions count as Copilot Chat requests
- IDE and CLI access needs the GitHub MCP server with the `copilot_spaces` toolset
- In the IDE, whole-repository and uploaded sources **aren't** available

Check current docs. This area changes.

Repository instructions

`.github/copilot-instructions.md`

- Added to **every** request in the repository
- Short rules for style, build, test, and what to avoid
- Lives in git, so the team shares it

Also read by Copilot: `AGENTS.md`, and user-level files in `~/.copilot/`.

Path-specific instructions

`.github/instructions/routes.instructions.md`

```
---
applyTo: "src/routes/**/*.js"
---

Register fixed paths before `/:id` routes.
```

- Loads only when Copilot works on a **matching file**
- Comma-separate globs: `**/*.ts,**/*.tsx`
- Keeps the always-on file small

Prompt files

`.github/prompts/low-stock.prompt.md`

- A **saved prompt** you run on purpose, usually as a slash command
- Good for a task you repeat: review, scaffold, write docs
- Support differs by tool. Check the docs for yours.

Instructions are always on. A prompt file runs when you call it.

Custom agents and skills

	Gives Copilot	Loads
Custom agent (`.agent.md`)	A role with its own tools and rules	When you pick it or it's delegated to
Skill (`SKILL.md`)	A repeatable workflow	When your request matches its description

Both are covered in Module 3. Here, know where they sit.

Memory

Copilot can **remember facts** from past work.

- **Repository facts:** conventions, commands. Shared with people who can use Memory in that repository.
- **User preferences:** how you like to work. Follow you across repositories.

Copilot writes them, not you. Review and delete what's wrong.

Available on paid plans. Check the docs for your surface.

@-mentions and file context

The quickest context: **point at it**.

- Attach a file, selection, or folder in chat
- In the CLI, `@path/to/file`
- In the IDE, `#file` and open tabs

Good for one task. Gone when the chat ends.

Comparison

Mechanism	Scope	Loads	Best for
Space	Anyone you share it with	You ask in it	Reference docs and specs
Repo instructions	Whole repository	Always	House rules
Path instructions	Matching files	When working on them	Rules for one area
Prompt file	Repository or user	You run it	Repeated tasks
Custom agent	Repository or user	You pick it	A role and tool set
Skill	Repository or user	Matches the request	A workflow with files
Memory	Repo or you	Automatically	Learned facts
@-mention	This chat	You attach it	One-off context

Which one when

You want to…	Use
Answer questions from your spec and docs	**Space**
Enforce "ES modules, `node:test`" everywhere	**Repo instructions**
Apply a rule only to `src/routes/`	**Path instructions**
Rerun the same review prompt	**Prompt file**
Look at one file right now	**@-mention**

Facts the code lacks go to a **Space**. Habits for every task go to **instructions**.

Demo: one task, three contexts

Task: add `GET /api/products/low-stock` with tests.

1. No context
2. With the Space
3. With instructions

We score each on the same checklist.

Pros and cons of a Space

Pros	Cons
Context outlives chat	**Stale** if sources aren't kept up
Shareable	**Too much** context hurts answers
Follows `main`	A bad push changes answers
Right sources, not the whole repo	**Sensitive data** can leak to viewers

Tips

- Start with 3 to 5 sources.
- Prefer **files** for key docs. Use a repository for big doc trees.
- Write Space instructions as the job and its limits.
- Keep instructions under about a page. Cut what code already shows.
- Test any context with a question you know the answer to.
- Date-check sources the way you'd date-check docs.

Common mistakes

Mistake	Fix
Adding a whole repository "just in case"	Add the files that matter
One giant instructions file	Split with `applyTo`
Instructions that contradict the code	Fix the instruction or the code
Secrets or customer data in a shared Space	Synthetic or public data only
Uploads that go stale	Link GitHub files instead
Blaming the model for missing facts	Check what context it had

Lab

Build a Space. Write instructions. Run one task four times.

Part	Time
Set up	15 min
Build the Space	25 min
No context, then Space	25 min
Write instructions	20 min
Instructions, then both	25 min
Compare	10 min